Organização de Computadores RISC-V

Interrupções Externas

Agenda

1. Introdução.
2. Revisando Pooling com espera ocupada.
3. Interrupções externas.
4. Interrupções em RV32i
5. Fluxo de manipulação de interrupções.
6. Comunicação via UART.

Revisando Acesso à Periféricos via Pooling

Acessando Periféricos via Pooling

- Na aula anterior discutimos como os programas podem interagir diretamente com dispositivos periféricos de entrada e saída, sem a participação do sistema operacional.
- Particularmente, discutimos **I/O mapeado por porta** e **I/O mapeado em memória**, bem como a interação baseada **Pooling** com **Espera Ocupada** (buzy waiting).
- Nesta aula discutiremos a técnicas interação baseada em Interrupção Externa.

Pooling

- Polling é um método no qual o programa é projetado de modo que a CPU verifique periodicamente se os periféricos precisam de atenção.
- Sempre que houver um periférico que precise de atenção, o programa invoca uma rotina para lidar com o periférico.
- O algoritmo ao lado ilustra um programa que emprega polling para lidar com um teclado hipotético.

```
while True do
    if keypadPressed() then
        k ← getKey() ;
        pushKeyOnQueue(k) ;
    end
    Compute() ;
end
```

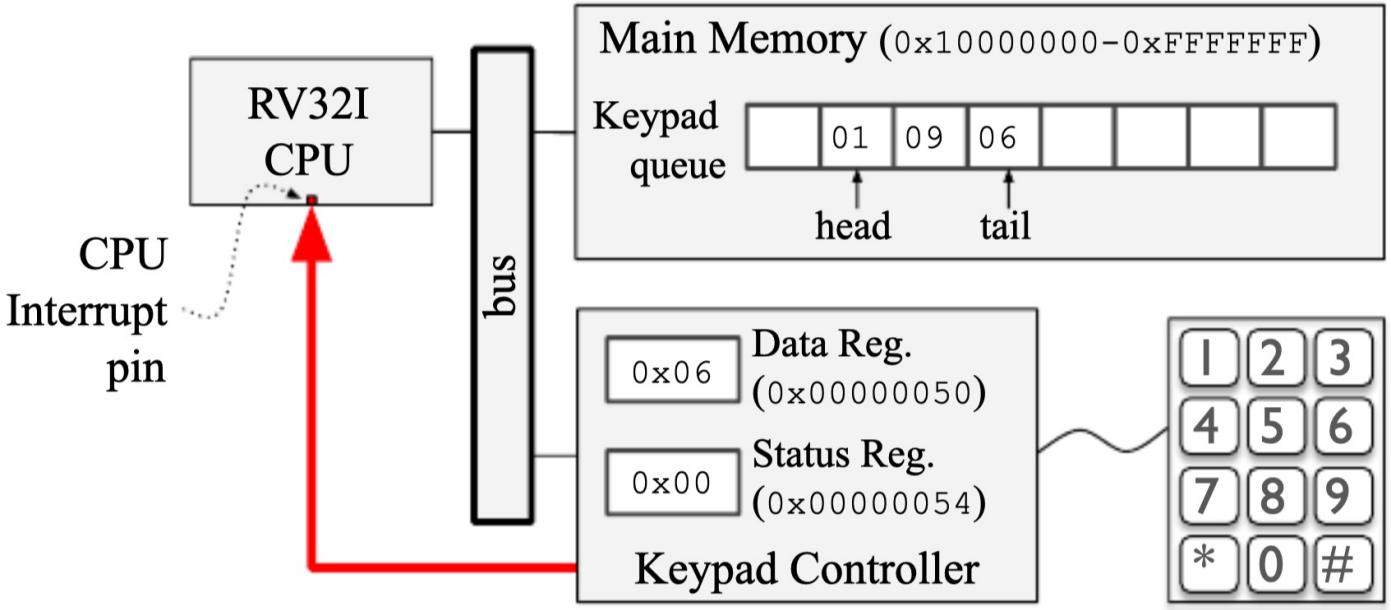
Pooling

- Observe que a quantidade de trabalho realizado pela rotina Compute() afeta a frequência com que o teclado é verificado.
- Por um lado, quanto mais tempo a rotina Compute() leva para ser executada, maior a chance de ocorrer perda de dados.
- Por outro lado, interromper a computação para que cada chamada a Compute() seja executada rapidamente (por exemplo, executando apenas uma pequena fração da computação cada vez que é invocada) pode causar uma grande sobrecarga (a verificação de periféricos pode levar muito tempo) e pode dificultar o projeto e a implementação do programa.
- Como consequência, a verificação por polling geralmente **não é** a melhor abordagem para verificar e lidar com eventos de periféricos.

Interrupções Externas

Interrupções Externas: Ilustração

- Nessa abordagem, a CPU possui um pino de interrupção externa que informa à CPU se há ou não uma interrupção externa pendente.
- No exemplo, sempre que uma tecla é pressionada, o controlador envia um sinal à CPU pelo pino de interrupção.
- O hardware da CPU monitora constantemente o pino de interrupção como parte do ciclo de execução de instruções. Quando recebe uma sinal, o fluxo de execução é interrompido para executar a rotina de serviço de interrupção (ISR), que é uma rotina de software que gerencia a interrupção.



Interrupções Externas

- Portanto, interrupções externas consistem em um mecanismo que permite que o hardware informe à CPU que precisa de atenção.
- Nessa abordagem, o periférico envia um sinal de interrupção para a CPU e, assim que a CPU recebe esse sinal, ela:
  1. Salva o **contexto** do programa que está em execução.
  2. Invoca a rotina de serviço da interrupção (ISR).
  3. Restaura o contexto do programa salvo e continua a execução.
- Existem várias maneiras de implementar ISRs, mas, em geral, elas costumam salvar o contexto do programa em execução (e.g., o conteúdo dos registradores da CPU) na memória principal, interagem com o periférico que enviou o sinal de interrupção e, finalmente, restauram o contexto salvo para que a CPU continue executando o programa que foi interrompido.

Detectando Interrupções Externas

Adaptando o ciclo de execução de instruções da CPU para lidar com interrupções.

```
while True do
    /* Check for interrupts */
    if (interrupt_pin = '1') and (interrupts_enabled = '1') then
        /* Invoke the ISR */
        SAVED_PC ← PC ;
        PC ← ISR_ADDRESS;
        interrupts_enabled ← '0';
    end
    /* Fetch instruction and update PC */
    IR ← MainMemory[PC] ;
    PC ← PC+4;
    ExecuteInstruction(IR);
end
```

Invocando a Rotina de Serviço de Interrupção Adequada

- Um sistema de computação pode conter vários periféricos que podem interromper a CPU. Além disso, cada periférico geralmente requer uma rotina especializada para lidar com suas interrupções. Por exemplo, um controlador de teclado pode exigir uma rotina que adicione o valor do registrador de dados a uma fila na memória principal, enquanto um controlador de mouse pode precisar de outra ação.

- Como consequência, sempre que ocorre uma interrupção, o sistema precisa executar duas tarefas: (i) identificar qual periférico interrompeu a CPU e (ii) invocar a rotina apropriada para lidar com a interrupção.

- Dependendo da arquitetura do sistema, essas duas tarefas podem ser executadas por hardware, software ou uma combinação de ambos. De fato, existem várias maneiras de identificar qual periférico interrompeu a CPU e invocar a rotina apropriada para lidar com a interrupção.

Interrupções no RV32i

Interrupções Externas no RISC-V

- No RISC-V, o processo é bem definido e dividido entre o que o hardware faz automaticamente e o que o software (ISR) precisa fazer. Visão geral do fluxo de tratamento de interrupção externa no RISC-V:
  1. Hardware salva parte do contexto automaticamente.
  2. Hardware desvia para o endereço da ISR (Interrupt Service Routine).
  3. Software salva o restante do contexto (se necessário).
  4. Software atende a interrupção.
  5. Software restaura o contexto.
  6. Hardware retorna ao programa interrompido.
- Na sequência, vamos detalhar esses passos.

Geração da Interrupção

- Cada periférico gera uma interrupção ativando uma linha física que está conectada ao controlador de interrupções.
- Por exemplo:
  - **UART** —— linha 10 ——► PLIC (Plataform-Level Interrupt Controller)
  - TIMER —— linha 7 ——► PLIC (Plataform-Level Interrupt Controller)
  - GPIO —— linha 3 ——► PLIC (Plataform-Level Interrupt Controller)
  - DISK —— linha 12 ——► PLIC (Plataform-Level Interrupt Controller)

- Os Números associados à linha pelo PLIC são definidos pela plataforme, no caso 10 é o número definido para UART no QEMU virt.

PLIC Envia Sinal de Interrupção para a CPU

- A interrupção será aceita se estiver habilitada:
  - `mie.MEIE = 1  # enable external interrupt`
  - `mstatus.MIE = 1  # global enable`

- Se estiver habilitada, então o bit `MEIP` (Machine External Interrupt Pending) registrador CSR `mip` (Machine Interrupt Pending) é ativado para indicar que há uma interrupção externa pendente.
- Com esse bit ativado, a CPU detecta que há interrupção externa pendente como parte do ciclo de execução de instrução.

Tratamento da Interrupção Externa

- Quando a CPU aceita a interrupção, o hardware executa automaticamente as seguintes ações:
  1. Salva o ponto de retorno em `mepc`: mepc ← PC atual.
  2. Desabilita temporariamente interrupções e salva o modo de execução atual da CPU:

     1. `mstatus.MPIE ← mstatus.MIE.`
     2. `mstatus.MIE ← 0.`
     3. `mstatus.MPP ← modo atual.`
  3. Salva o motivo da interrupção em `mcause`.

Tratamento da Interrupção Externa

- Desvio para o handler:
  - `PC` ← `mtvec`. Obviamente, em código bare metal, você deve configurar `mtvec.`
- O processador segue a execução no handler (ISR) que identifica a interrupção verificando `mcause`.
- No caso de interrupção externa, PLIC é consultado para obter o id da interrupção.
- ISR executa o serviço solicitado e em seguida informa PLIC que terminou.
- ISR retorna com `mret.`